



SOICT

PROJECT REPORT

US Flight Analysis

Course: IT4043E

Supervisor: Dr. Tran Viet Trung

Authors:

Phạm Long Hải
Phạm Công Hiếu
Trần Vương Hưng
Vũ Việt Long
Nguyễn Phan Thắng

Student ID:

20225492
20225493
20225496
20225508
20225529

Hanoi, December 2025

Table of Contents

1 Introduction	3
1.1 Problem Statement	3
1.2 Project goals	3
1.3 Data overview	3
1.4 Deployment strategy	4
1.5 Scope and limitation	4
2 Architecture and Design	5
2.1 General Architecture	5
2.2 Architecture components	5
2.2.1 Batch layer	5
2.2.2 Speed Layer	6
2.2.3 Serving layer	6
2.2.4 Data flow and component interaction diagrams	7
2.2.5 Kafka components	7
2.2.6 Spark components	8
2.2.7 HDFS Components	8
3 Implementation Details	9
3.1 Full code repo	9
3.2 Cluster setup	9
3.3 Batch Layer Implementation	10
3.4 Speed Layer Implementation	11
3.5 Serving Layer Implementation	12
4 Lesson Learned	14
4.1 Lesson 1: Managing Duplicates and Versioning with Iceberg	14
4.1.1 Problem Description	14
4.1.2 Approaches Tried	15
4.1.3 Final Solution:	15
4.2 Lesson 2: Resource Allocation and Metadata-Driven Querying	15
4.2.1 Problem Description	15
4.2.2 Approaches Tried	15
4.2.3 Final Solution	15
4.3 Lesson 3: State Management and Exactly-Once Recovery	16
4.4 Lesson 4: Service Discovery via Network Tunneling	16
4.4.1 Problem Description	16
4.4.2 Approaches Tried	16
4.4.3 Final Solution	16
5 Conclusion and Future work	17
5.1 Conclusion	17

5.2 Future works 17

1 Introduction

1.1 Problem Statement

Air transportation is one of the most important industries, playing a crucial role in promoting economic growth and strengthening global connectivity. However, challenges such as flight delays and cancellations create significant inconvenience for passengers, lead to financial losses for airlines, and negatively affect the overall operational efficiency of the aviation industry.

Various factors, including weather conditions, technical issues, airport operations, and air traffic management, can increase the likelihood of delays or cancellations. As a result, analyzing flight data to identify trends and root causes, as well as to anticipate potential disruptions, is essential for improving passenger experience and optimizing the performance and reliability of the aviation sector.

1.2 Project goals

Big Data technologies and real-time processing play a critical role in enabling continuous monitoring and faster, more informed decision-making within the aviation industry. By leveraging real-time analytics, airlines and airport operators can detect potential delays at an early stage and respond proactively, reducing their impact on flight schedules. These technologies also help optimize flight operations and improve resource allocation, including aircraft utilization, crew scheduling, and airport facilities, ultimately enhancing overall system efficiency and passenger satisfaction.

In this project, streaming flight data is simulated to reflect real-world aviation operations and analyzed using a Lambda architecture. This architecture combines batch processing for comprehensive historical analysis with stream processing for real-time insights, enabling efficient, scalable, and accurate data analysis. The integrated approach ensures timely detection of issues while maintaining analytical depth, supporting better operational decisions and improved performance across the aviation system.

1.3 Data overview

The dataset for this project is the Airline Delay Analysis Dataset from the US Bureau of Transportation Statistic (the download website: https://www.transtats.bts.gov/DL_SelectFields.aspx?gnoyr_VQ=FGJ&QO_fu146_anzr=b0-gvzr). The dataset contains flight data for all US' domestic flight performance, focusing on the time delay of the flights.

The dataset contains data from 2020 to June of 2025, with each month saved in its own .csv file. We have uploaded the dataset to Kaggle here: <https://www.kaggle.com/datasets/hieupc/flight-data-from-us-bts>

The dataset consists of 26 tables indicated as such

Field	Description
FL_DATE	Flight date (m/d/yyyy hh:mm:ss format)
OP_UNIQUE_CARRIER	Unique carrier code
OP_CARRIER_FL_NUM	Flight number
TAIL_NUM	Aircraft tail number
ORIGIN	Origin airport code
ORIGIN_CITY_NAME	Origin airport city name
DEST	Destination airport code
DEST_CITY_NAME	Destination airport city name
DEP_TIME	Actual departure time (local time, hhmm)
DEP_DELAY	Difference in minutes between scheduled and actual departure (negative = early)
TAXI_OUT	Taxi-out time (minutes)
WHEELS_OFF	Wheels-off time (local time, hhmm)
WHEELS_ON	Wheels-on time (local time, hhmm)
TAXI_IN	Taxi-in time (minutes)
ARR_TIME	Actual arrival time (local time, hhmm)
ARR_DELAY	Difference in minutes between scheduled and actual arrival (negative = early)
AIR_TIME	Flight time (minutes)
DISTANCE	Distance between airports (miles)
CANCELLED	Cancelled flight indicator (1 = Yes)
DIVERTED	Diverted flight indicator (1 = Yes)
CARRIER_DELAY	Carrier delay (minutes)
WEATHER_DELAY	Weather delay (minutes)
NAS_DELAY	National Air System delay (minutes)
SECURITY_DELAY	Security delay (minutes)
LATE_AIRCRAFT_DELAY	Late aircraft delay (minutes)
CANCELLATION_CODE	Cancellation reason (A=Carrier; B=Weather; C=NAS; D=Security)

Table 1: Flight dataset attributes (2018–2025)

1.4 Deployment strategy

The project deployment strategy is listed as followed:

- **Phase 1 - Data Acquisition:** Developed Python scripts to get the data from the US Bureau of Transportation Statistic.
- **Phase 2 - Docker Compose:** Deployed the full stack using docker-compose.yaml to verify component connectivity (networking, volumes) in a controlled environment.
- **Phase 3 - Kubernetes Migration:** Converted Docker Compose services to K8s environment. Used Helm for managing the complex Kafka installation, and give more calculation and analysis, working on the full data.

1.5 Scope and limitation

In this project our group try to apply Big Data technique for the US Flight Analysis, one of the big problem is handling big data, our data consists of almost 5 millions row, which makes processing and working in the big data architecture cost a lot of memories which limit the analysis our group can do and work with.

2 Architecture and Design

2.1 General Architecture

This project adopts the Lambda Architecture, a powerful data-processing design model that integrates both batch processing and real-time (stream) processing. This architecture is particularly well suited for applications that require real-time data analytics while still ensuring high accuracy and completeness when analyzing historical data. By combining these two processing paradigms, the system can deliver timely insights without sacrificing the reliability of long-term analytical results.

One of the key advantages of using Lambda Architecture in this project is its scalability. The system is capable of efficiently handling a large-scale flight dataset of approximately 4 GB, and it is designed to seamlessly scale as data volume increases, whether from historical records or continuous real-time streams. In addition, the architecture provides strong fault tolerance by separating responsibilities across independent processing layers. This layered design ensures system stability and resilience, allowing the platform to continue operating reliably even when individual components encounter failures.

Furthermore, Lambda Architecture achieves an effective balance between low latency and high accuracy. The real-time processing layer enables rapid analysis and immediate insights from incoming flight data, while the batch layer performs comprehensive and precise computations on historical data. Together, these components form a robust and end-to-end analytical system that not only supports accurate prediction models but also delivers timely, detailed information to aid critical decision-making processes.

2.2 Architecture components

2.2.1 Batch layer

In this project, historical flight data is uploaded to HDFS through Apache Spark by transforming local CSV files into a structured Apache Iceberg table, where HDFS serves as the underlying storage layer while Iceberg provides transactional guarantees, schema enforcement, and efficient partitioned storage.

Spark then performs large-scale aggregations across the entire dataset, enabling accurate analysis of long-term trends that cannot be reliably captured through real-time processing alone.

The results of these analyses are written back to HDFS as Iceberg “gold” tables, replacing previous versions atomically. This approach ensures that downstream consumers always access a consistent and up-to-date analytical view, while also benefiting from Iceberg features such as schema evolution and snapshot-based versioning. Overall, the batch

analysis layer provides reliable, high-accuracy insights derived from complete historical data, complementing the real-time processing layer within the Lambda Architecture.

2.2.2 Speed Layer

The streaming producer is responsible for simulating and publishing real-time flight events to Apache Kafka, which serves as the ingestion layer for the system's streaming pipeline. Kafka receives and manage the streaming data flow from sources.

Spark Streaming processes these incoming data streams in real time, applying continuous computations and analytical operations as soon as new events arrive. Instead of waiting for data to be stored and processed in large batches, Spark Streaming ingests records incrementally and performs transformations such as filtering, aggregation, and enrichment with very low latency. This enables the system to react quickly to changes in flight conditions, delays, or operational anomalies as they occur.

By leveraging Spark's distributed processing engine, the streaming layer can scale horizontally to handle high-throughput data while maintaining consistent performance. The computed results—such as real-time delay statistics or anomaly indicators—can be immediately forwarded to downstream systems, dashboards, or alerting services. As a result, Spark Streaming plays a crucial role in delivering timely, actionable insights that support rapid decision-making and complement the deeper, more accurate analysis produced by the batch layer.

2.2.3 Serving layer

The Serving Layer acts as the central interface, transforming complex datasets from the Lakehouse architecture into interactive insights that support strategic decision-making. In this project, Streamlit is utilized as the primary framework for building the dashboard, integrated with Apache Iceberg and PySpark to query data directly from HDFS. By leveraging the Iceberg table format at this layer, the system ensures ACID consistency during concurrent read/write operations, allowing the dashboard to access processed “Gold Tables” with high reliability and speed. To optimize performance, the serving layer implements intelligent Caching mechanisms, maintaining stable Spark sessions and reducing system overhead when retrieving repetitive historical analytics.

The interface of the Serving Layer is designed with two distinct modules to serve different analytical purposes. The Real-time Monitoring module utilizes data from the Speed Layer to provide an instantaneous view of flight operations. Through an auto-refresh mechanism triggered every 10 seconds, the dashboard displays dynamic KPI metrics such as average system delay, active flight counts, and real-time hub activity. A standout feature is the “Live Log” table, which allows users to directly track the data stream as it is ingested from Kafka into HDFS, providing full transparency of the system's operational status at any given moment.

Simultaneously, the Historical Insights module focuses on extracting deep-dive value from the Batch Layer. At this stage, the Serving Layer visualizes long-term trends using line charts and box plots, helping to identify root causes of delays such as weather, carrier issues, or National Airspace System (NAS) congestion. Users can easily compare performance across different airlines, evaluate airport congestion through taxi-in/out metrics,

and identify routes with the highest cancellation rates. This synergy between immediate responsiveness and in-depth historical analysis creates a comprehensive Serving Layer that flexibly meets all data exploitation needs in aviation management.

2.2.4 Data flow and component interaction diagrams

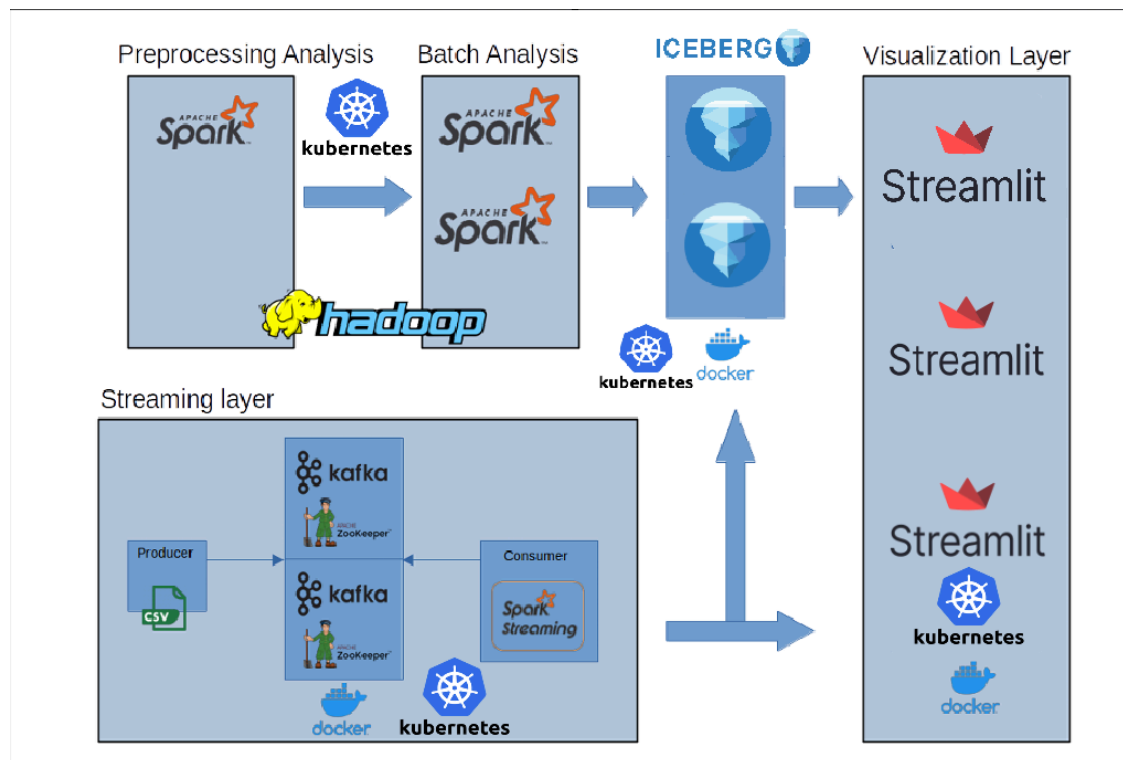


Figure 1: Pipeline Architecture

2.2.5 Kafka components

In this implementation, Apache Kafka is deployed in ZooKeeper mode, where ZooKeeper is responsible for managing cluster metadata, broker registration, and controller election. The Kafka cluster consists of a single ZooKeeper pod and a single Kafka broker pod, which is sufficient for development and functional testing purposes. Since only one broker is present, it automatically assumes the controller role, as controller election in ZooKeeper-based Kafka occurs among the available brokers.

The Kafka broker is deployed using the confluentinc/cp-kafka:7.6.0 image and is configured with a dual-listener architecture to support both internal and external communication. An internal listener is exposed on port 29092, allowing in-cluster components such as Apache Spark to communicate with Kafka using the service name kafka. An external listener is exposed on port 9092, enabling access from outside the Kubernetes cluster through port forwarding, which allows external applications such as a Python-based producer to connect to the Kafka broker.

A Kubernetes Service is used to expose both internal and external Kafka ports. Communication between the Kafka broker and ZooKeeper is established via the endpoint zookeeper:2181, ensuring proper coordination and metadata management. Due to the single-broker configuration, the Kafka cluster operates with a replication factor of one, meaning that partitions are not replicated across multiple nodes. While this limits fault tolerance, it is acceptable for non-production environments.

For simplicity during development, all Kafka communication protocols are configured to use PLAINTEXT, avoiding encryption and authentication overhead. Although this configuration eases setup and debugging, it does not provide secure communication. In a production environment, this deployment should be extended to multiple brokers and enhanced with SSL/TLS encryption and appropriate security mechanisms to ensure reliability and data protection.

2.2.6 Spark components

Apache Spark is deployed in standalone cluster mode within a Kubernetes environment and is integrated with Apache Iceberg for analytical workloads stored on HDFS. The Spark application initializes a SparkSession configured with the Iceberg Spark extensions and the Hadoop-based Iceberg catalog. The catalog warehouse is explicitly set to an HDFS-backed directory, and the Hadoop fs.defaultFS parameter points to the HDFS NameNode, ensuring that all table metadata and data files are persisted in HDFS.

Iceberg support is enabled by loading the iceberg-spark-runtime-3.5_2.12 package at runtime. This configuration allows Spark SQL to natively create, query, and manage Iceberg tables using transactional semantics while leveraging HDFS as the underlying storage layer. As a result, Spark jobs can perform scalable analytical queries with schema evolution and snapshot isolation.

The Spark cluster follows a master-worker architecture. A Spark Master pod is deployed and exposed via a Kubernetes Service, providing the cluster entry point on port 7077 and a web-based monitoring interface on port 8080. The Spark Master is responsible for resource scheduling, worker registration, and job coordination. Two Spark Worker pods are deployed and configured to automatically connect to the master using the internal service address spark:

In addition to the master and worker nodes, a dedicated Spark client pod is deployed. This pod serves as an interactive or submission environment from which Spark jobs can be launched into the cluster. The client container runs continuously and shares the same Spark and Iceberg runtime image as the master and workers, ensuring configuration consistency across all Spark components.

Overall, this deployment enables a fully containerized Spark analytics platform that integrates seamlessly with HDFS and Iceberg. The architecture supports distributed execution, centralized resource management, and reliable table storage, making it suitable for batch analytics and exploratory data processing in a development or research environment.

2.2.7 HDFS Components

HDFS is deployed within the Kubernetes environment to provide a reliable and scalable storage layer for analytical workloads. The system consists of a NameNode and one or more DataNodes. The NameNode is responsible for managing file system metadata, including directory structure, file permissions, and block locations, while DataNodes store the actual data blocks and handle read and write operations. Spark and other clients interact with HDFS through the NameNode, which coordinates access to data stored across the DataNodes.

To ensure metadata durability, the NameNode uses a Kubernetes PersistentVolumeClaim named hdfs-storage with a capacity of 10 GB, mounted at the NameNode's storage directory. This persistent volume preserves critical filesystem metadata across pod restarts. Through this configuration, HDFS serves as the primary distributed storage back-end for

Spark and Iceberg, enabling scalable and fault-tolerant storage of historical data in the batch processing layer.

3 Implementation Details

3.1 Full code repo

The full code for the project can be found in the Github repo: <https://github.com/Cherricott/Bigdata>, at branch `release1.0`.

3.2 Cluster setup

To deploy the application on Kubernetes, the process begins by executing the script `./setup-registry.sh`. This script initializes a local Kubernetes environment using Kind and configures a private Docker registry accessible at `localhost:5001`. The registry allows locally built container images to be stored and pulled by Kubernetes pods without relying on external image repositories, ensuring a fully self-contained development setup.

Once the registry and cluster are available, the Spark application image is built locally using Docker and tagged as `localhost:5001/spark-iceberg-app:latest`. This tag explicitly points to the local registry created in the previous step. After the image is built, it is pushed to the local registry so that it becomes accessible to all Kubernetes nodes within the cluster.

The deployment phase is then carried out using a sequence of Kubernetes manifests applied in a logical order. First, `01-storage.yaml` is applied to provision persistent storage resources, such as `PersistentVolumeClaims`, which are required for components like HDFS to store data and metadata reliably. Next, `02-infra.yaml` deploys the core infrastructure services, including distributed system components such as Kafka, and ZooKeeper, which form the foundation of the data processing pipeline. Finally, `03-spark-cluster.yaml` deploys the Apache Spark cluster, including the Spark Master, Spark Workers, and the Spark client, enabling batch and analytical workloads to be executed on top of the previously deployed storage and infrastructure layers.

The detailed on how to run set up can be seen in our repo's `README.md` file

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: hdfs-storage
5  spec:
6    accessModes:
7      - ReadWriteOnce
8    resources:
9      requests:
10       storage: 10Gi
```

Figure 2: Example: 1-storage.yaml

At the end, if everything is run correctly you can check that all of the nodes are working and running like in this image

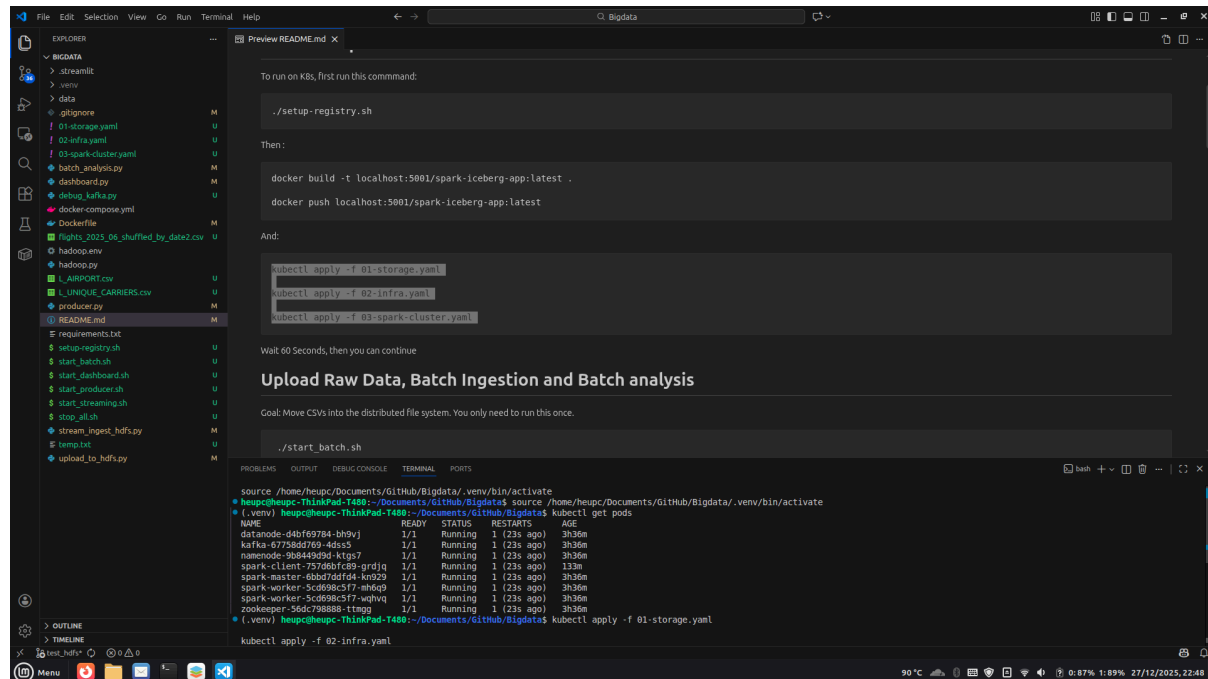


Figure 3: Example of everything working and running

3.3 Batch Layer Implementation

In order to upload historical data into a batch layer, we need to run the script `upload_to_hdfs.py`.

This script performs a batch upload of historical flight data into HDFS using Apache Spark with Apache Iceberg as the table format. First, a `SparkSession` is initialized with Iceberg extensions and a Hadoop-based catalog. The Iceberg warehouse directory is explicitly configured to reside in HDFS (`namenode:9000/warehouse`), and the HDFS NameNode is set as the default filesystem. This ensures that both table metadata and data files are stored persistently in HDFS.)

Before ingesting any data, the script initializes the Iceberg infrastructure by creating a namespace and a table if they do not already exist. This step is critical because HDFS may be reset between runs; recreating the namespace and table guarantees that the ingestion process can proceed safely. The Iceberg table `history_flights` is defined with all 26 columns and is partitioned by year derived from the flight date, which improves query performance for time-based analytics.

For each year, Spark reads the CSV data, selects and casts the required columns to match the Iceberg table schema, and converts the flight date into a proper `DATE` type. Rows with invalid or unseparable dates are filtered out to ensure data quality. Once the data is aligned with the target schema, it is appended to the Iceberg table using Iceberg's transactional write mechanism.

As a result, Spark writes the processed data as Iceberg-managed files into HDFS, while Iceberg maintains metadata such as snapshots and manifests. This approach guarantees

atomic writes, schema consistency, and reliable storage of historical flight data in HDFS, enabling efficient batch analytics and seamless integration with future streaming data.

For batch analysis, we run the script `batch_analysis.py`. The script read the Iceberg table we created beforehand and using Apache Spark, doing the complex aggregation and calculation in order to do the analysis on the historical flights data, for example: What is the top 5 worst airline...With each analysis, Sparks create and stored the data into another and different Iceberg table for visualization

```
# =====
# 7 ANALYSIS 5 & 6: EFFICIENCY & CANCELLATIONS
# =====

print("\n--- 5. Generating Airport Efficiency Stats ---")
# 1. Force drop the metadata from the catalog and HDFS
spark.sql("DROP TABLE IF EXISTS local.flight_stream.airport_efficiency PURGE")

airport_efficiency = (
    df_clean.groupBy("ORIGIN", "year", "month")
    .agg(
        avg("TAXI_OUT").alias("avg_taxi_out"),
        avg("TAXI_IN").alias("avg_taxi_in"),
        count("*").alias("flight_volume")
    )
    .filter(col("flight_volume") > 100)
    .orderBy(desc("avg_taxi_out"))
)

# 2. Use .create() instead of createOrReplace()
airport_efficiency.coalesce(1).writeTo("local.flight_stream.airport_efficiency").create()
print("✅ Saved: airport_efficiency")
```

Figure 4: Example of an analysis and saving it to a table

3.4 Speed Layer Implementation

For the speed layer, we have 2 main code a `producer.py` and a `stream_ingest_hdfs.py`. Running either one first is fine, but for the standard procedure, we starts the `stream_ingest_hdfs.py` first

This application implements a real-time flight data ingestion pipeline using Apache Spark Structured Streaming, Apache Kafka, Apache Iceberg, and the Hadoop Distributed File System (HDFS). Flight event records are continuously consumed from a Kafka topic, parsed into a structured format using a predefined schema, and transformed to ensure data consistency and correctness. Key preprocessing steps include date conversion, handling missing numeric values, and casting categorical indicators such as cancellation and diversion flags into appropriate data types. These transformations ensure that the ingested data is reliable and ready for analytical workloads.

The processed data is stored in an Iceberg table managed on HDFS, providing transactional guarantees, schema enforcement, and efficient metadata management. The table is partitioned by flight date at a daily granularity, which significantly improves query performance for time-based analytics and dashboard applications. Iceberg's table abstraction also enables scalable storage management and simplifies downstream data access through Spark SQL.

The script `producer.py` is responsible for streaming and simulating real live data. The producer sends data into a json and encode it to UTF-8 before sending to Kafka. Kafka

catches the streaming data and stored it into HDFS, and fit it into the Iceberg table created by the `stream_ingest_hdfs.py`. Messages are produced sequentially with a fixed time interval between sends, effectively emulating the behavior of streaming flight data rather than bulk ingestion. This controlled replay mechanism enables realistic testing of downstream streaming applications, such as Spark Structured Streaming consumers, under near-real-time conditions.

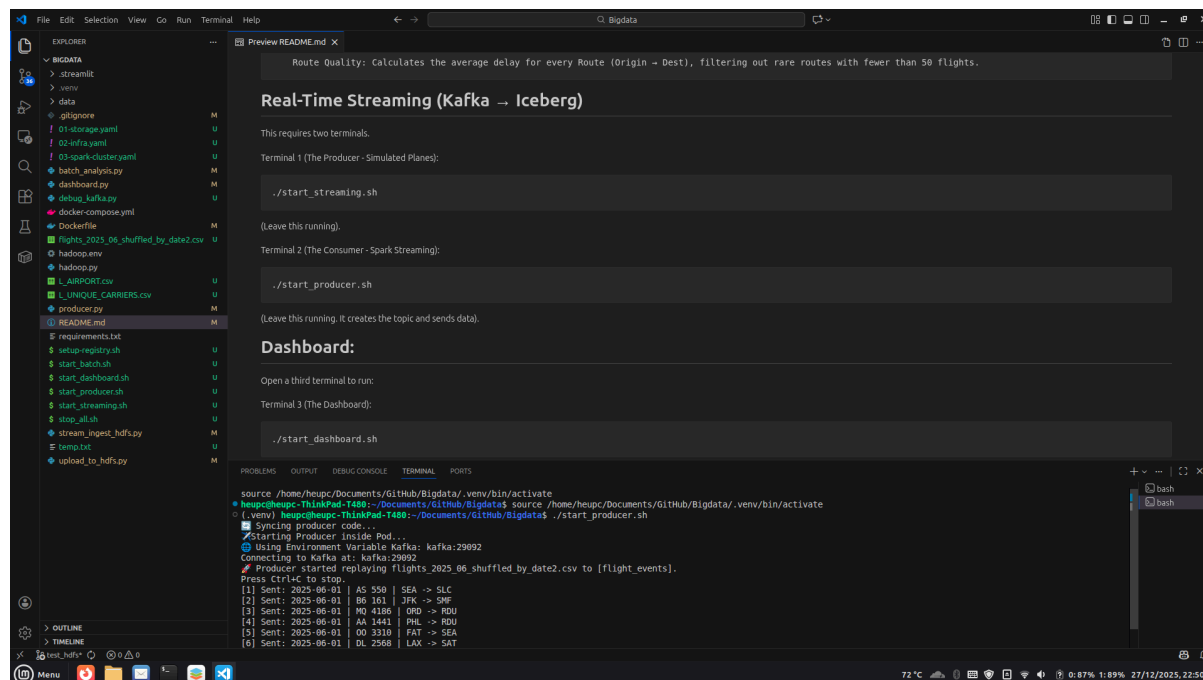


Figure 5: Producer running

3.5 Serving Layer Implementation

The implementation of the Serving Layer is centered on bridging the gap between big data storage and end-user visualization through a Python-native stack. At its core, the system initializes a dedicated `SparkSession` configured with Iceberg extensions and Hadoop configurations to enable seamless communication with the HDFS NameNode. To ensure high performance, the implementation utilizes caching decorators: **`st.cache_resource`** is used to maintain a persistent Spark connection, while **`st.cache_data`** is applied to historical queries with a defined Time-To-Live (TTL). This architecture prevents the overhead of re-initializing the Spark engine on every user interaction, allowing the dashboard to efficiently fetch and process data from the Iceberg “Gold Tables”.

The data retrieval logic is bifurcated to handle the distinct requirements of the Lambda architecture. For real-time monitoring, the system executes a **`refreshTable`** command to capture the latest snapshots committed by the Speed Layer, dynamically filtering for the most recent flight dates to calculate live KPIs. Meanwhile, the historical analysis module performs complex aggregations—such as calculating average delays per carrier and mapping cancellation codes to human-readable categories—directly on the processed datasets. These insights are then rendered through an interactive interface featuring Streamlit’s Tab component, which separates live telemetry from deep-dive historical trends.

To provide a truly “live” experience, the implementation integrates an automated synchronization loop using a combination of execution delays and the **`st.rerun()`** command. This

mechanism forces the dashboard to poll the Iceberg catalog every 10 seconds, ensuring that as new flight data is ingested via Kafka and processed by Spark Streaming, the visualizations are updated without manual intervention. From the use of **matplotlib** for statistical box plots to the real-time “Live Log” feed, the implementation ensures that the complex data lifecycle is presented to the end-user in a clear, responsive, and highly actionable format.

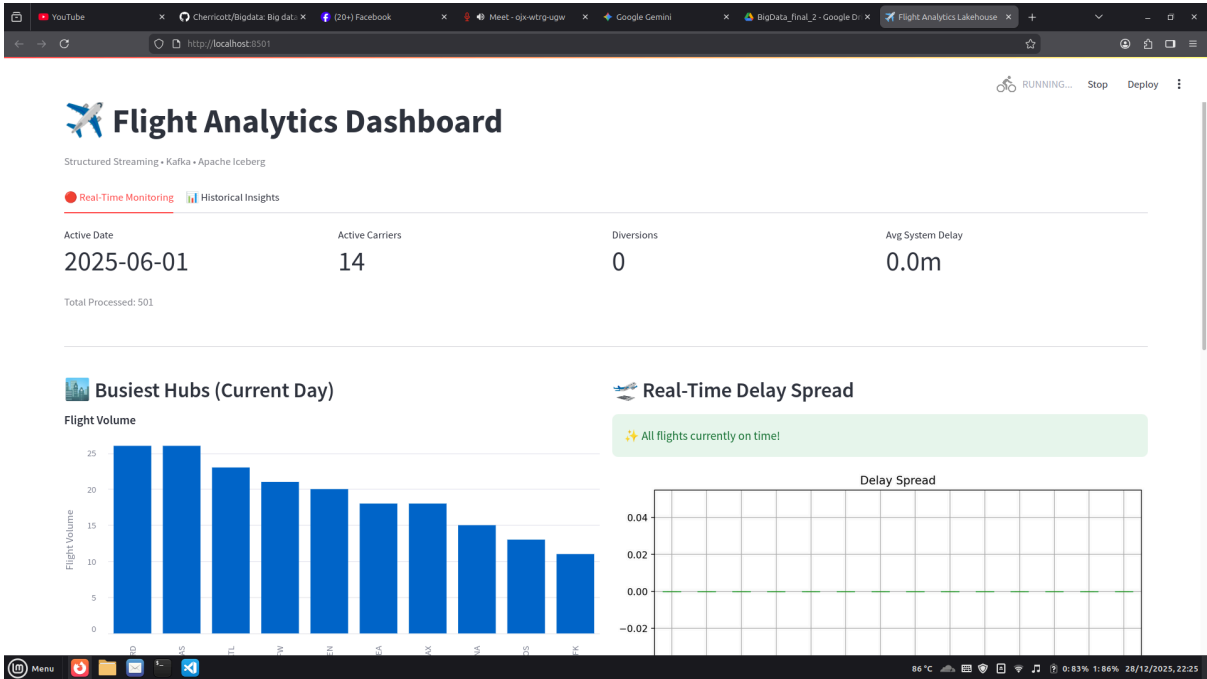


Figure 6: Live streaming data analysis

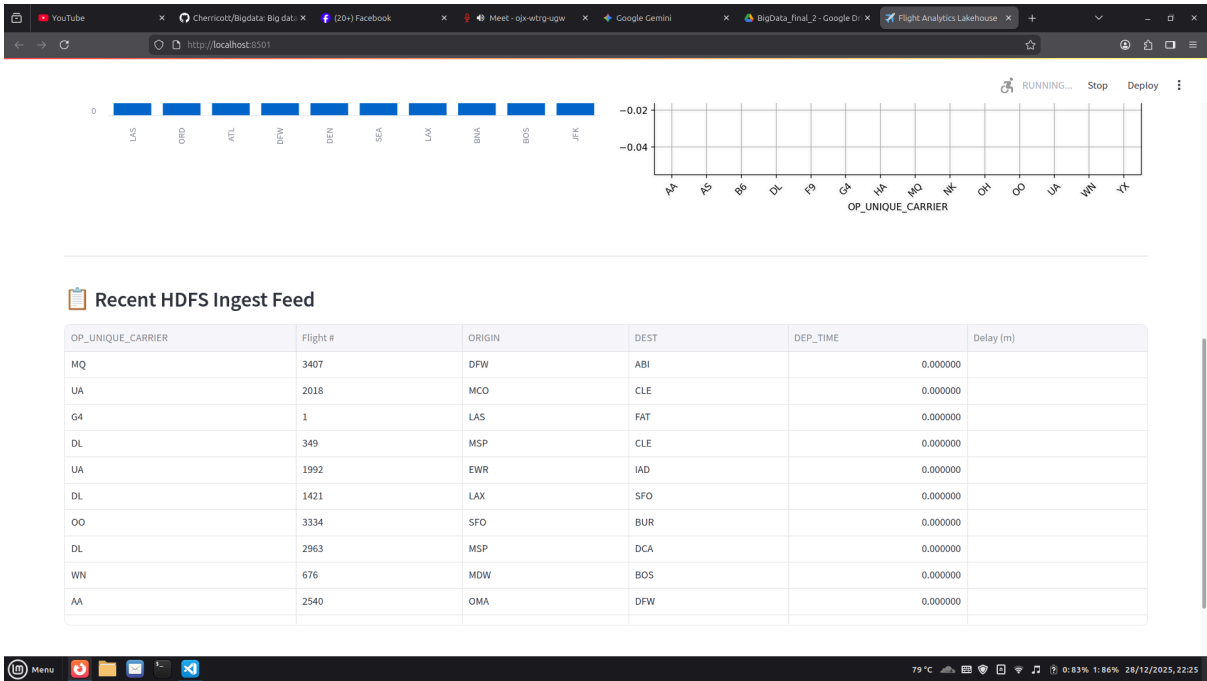


Figure 7: List of streaming data stored in HDFS

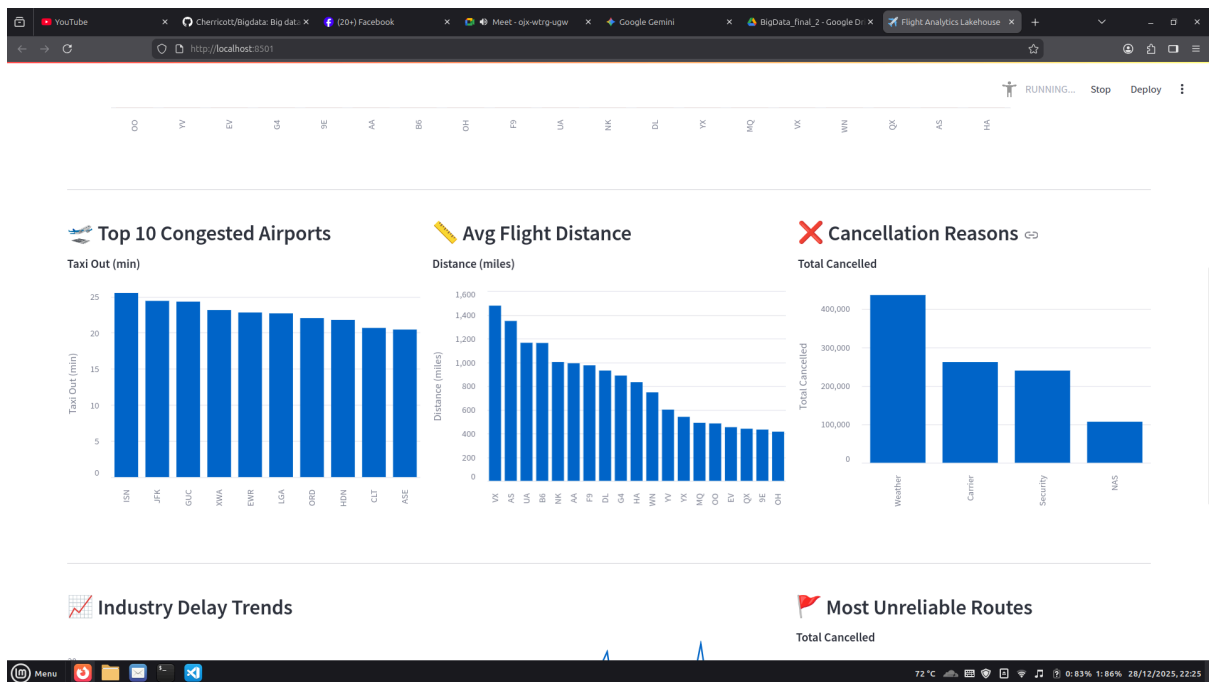


Figure 8: Visualization of batch analysis

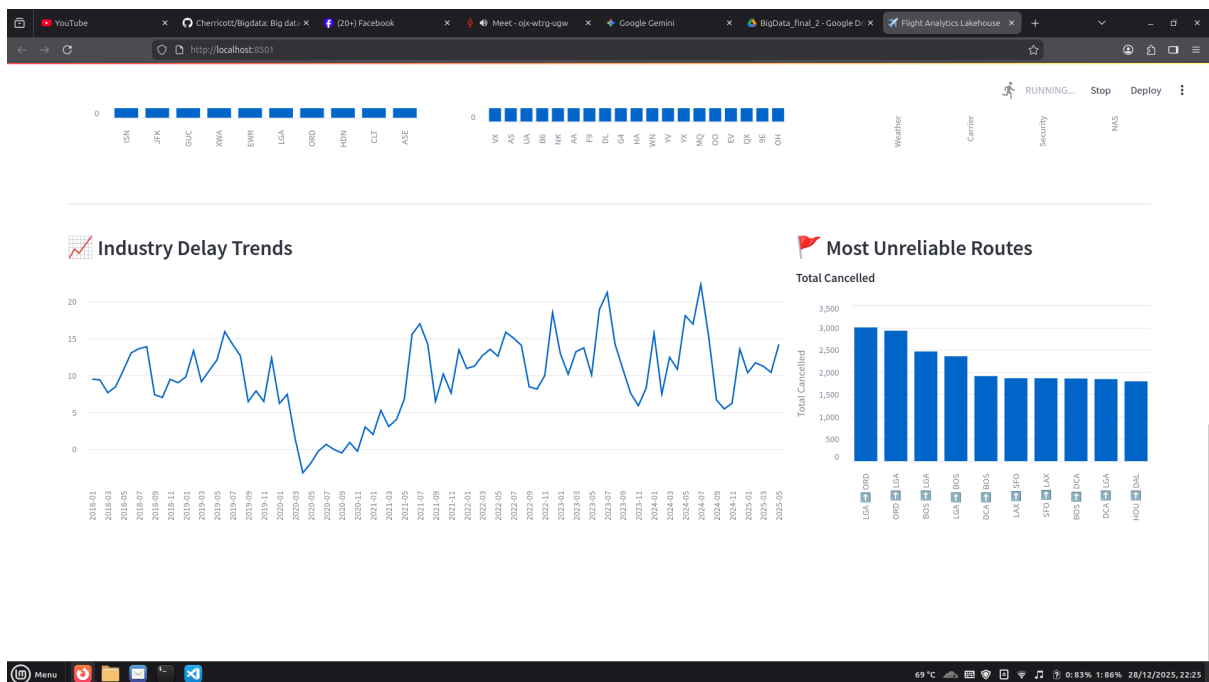


Figure 9: More visualization of batch analysis

4 Lesson Learned

4.1 Lesson 1: Managing Duplicates and Versioning with Iceberg

4.1.1 Problem Description

- **Context:** Managing a massive historical dataset (48.8M rows) alongside a high-velocity real-time Kafka stream.
- **Challenges:** Restarting the demo caused “double-counting” because the stream would re-process data already committed, or the historical batch would overlap with new incoming data.

- **System Impact:** Inaccurate dashboard metrics (double-counting flights) and storage bloat in HDFS.

4.1.2 Approaches Tried

- **Approach 1 (Standard Parquet):** Appending data to HDFS folders. Trade-off: No way to “undo” an ingestion without deleting the entire directory.
- **Approach 2 (Iceberg ACID Deletes):** Using a table format that supports SQL deletes. Trade-off: Requires more complex metadata management.

4.1.3 Final Solution:

- **Detailed Solution:** Migrated to Apache Iceberg using a Hadoop Catalog.
- **Implementation Details:** Integrated a pre-shutdown cleanup logic in `stop_all.sh` using: `DELETE FROM local.flight_stream.history_flights WHERE FL_DATE >= '2025-06-01';`.
- **Metrics and Results:** Achieved 100% data consistency across restarts with zero manual file manipulation. Reduced data footprint by 70% compared to raw CSV via Snappy compression.

Key Takeaways

- Use Table Formats (Iceberg/Delta) instead of raw files for pipelines requiring “idempotency.”
- Columnar storage (Parquet) is non-negotiable for high-volume analytical workloads.

4.2 Lesson 2: Resource Allocation and Metadata-Driven Querying

4.2.1 Problem Description

- **Context:** Querying 48.8M rows on a consumer-grade laptop.
- **Challenges:** Initial queries took over 30 seconds; Spark master crashed frequently due to OOM (Out of Memory) errors when trying to scan the entire HDFS warehouse.
- **System Impact:** High CPU latency and unstable cluster pods.

4.2.2 Approaches Tried

- **Approach 1 (High Parallelism):** Maxing out Spark cores. Trade-off: Overheated the host machine and caused Docker to hang.
- **Approach 2 (Metadata Pruning):** Leveraging Iceberg’s manifest files to skip data scans.

4.2.3 Final Solution

- **Detailed Solution:** Optimized Spark configurations for local resource constraints and used Iceberg metadata for “Data Skipping.”
- **Implementation Details:** Set `spark.driver.memory` and `spark.executor.memory` to specific caps; utilized Iceberg manifest files to avoid full directory scans.

- **Metrics and Results:** Query execution time for 48.8M rows dropped from 30+ seconds to 4.7 seconds.

Key Takeaways

- Metadata-heavy formats are more effective than “throwing more RAM” at a problem.
- On local clusters, resource caps are necessary to prevent the host OS from killing container processes.

4.3 Lesson 3: State Management and Exactly-Once Recovery

Problem Description

- **Context:** Real-time flight ingestion via Spark Structured Streaming and Kafka.
- **Challenges:** If the Spark client pod crashed, the “offset” (the marker of where it stopped reading Kafka) was lost.
- **System Impact:** Data gaps or massive duplicates upon recovery.

Approaches Tried

- **Approach 1 (Memory-only):** No checkpoints. Trade-off: Total loss of progress on every restart.
- **Approach 2 (Persistent Checkpointing):** Saving state to HDFS.

Final Solution

- **Detailed Solution:** Implemented HDFS-backed checkpointing.
- **Implementation Details:** Defined `checkpointLocation` pointing to hdfs:
- **Metrics and Results:** Successfully resumed streams exactly where they left off after multiple cluster reboots.

Key Takeaways · Checkpointing is the only way to achieve true Exactly-Once processing. · Explicitly clean checkpoints in reset scripts only when a “Fresh Demo” is required.

4.4 Lesson 4: Service Discovery via Network Tunneling

4.4.1 Problem Description

- **Context:** Connecting Local Python apps (Producer/Dashboard) to Containerized services.
- **Challenges:** Docker networks are isolated; external apps could not resolve `kafka:29092` or `namenode:9000`.
- **System Impact:** Constant “Connection Refused” errors during demo starts.

4.4.2 Approaches Tried

- **Approach 1 (Host Networking):** Running Docker in host mode. Trade-off: Incompatible with Kind clusters.
- **Approach 2 (K8s Port-Forwarding):** Creating explicit tunnels for each service.

4.4.3 Final Solution

- **Detailed Solution:** Standardized on K8s Service names for internal traffic and kubectl port-forward for external traffic.
- **Implementation Details:** Background tunnels were established for the Spark Master (8080) and Spark SQL (10000).
- **Metrics and Results:** Stable 3-way communication between local code, Kafka, and HDFS.

Key Takeaways

- Use logical service names for stability.
- Tunneling is the most reliable way to bridge local development tools with containerized infrastructure.

5 Conclusion and Future work

5.1 Conclusion

This project successfully demonstrated the deployment of a comprehensive Big Data pipeline on Kubernetes. We developed a Docker Compose setup and migrating to a K8s architecture. The system is capable of end-to-end data processing: ingesting weather data, processing it in real-time, archiving it to HDFS, and serving aggregated views via Streamlit.

5.2 Future works

In order to have more insight about Big Data concepts, we would want to try with some different design patterns to compare between different architectures.

Acknowledgements

This work is supervised and supported by Dr. Tran Viet Trung as part of the course Big Data Storage and Processing (IT4043E). In addition, the research and project received partial support from students of the School of Information and Communication Technology, Hanoi University of Science and Technology.